

NoteAI 智能笔记应用 - 技术方案设计书

一、项目概述

1. 基本信息

项目名称：NoteAI 智能笔记

项目类型：Android 原生移动应用

开发语言：Kotlin

最低兼容版本 (Min SDK)：API 24 (Android 7.0)

目标用户：需要高效记录、整理笔记，并希望利用 AI 辅助提效的学生、开发者及知识工作者。

2. 项目简介

NoteAI 是一款基于 MVVM 架构 开发的现代化 Android 智能笔记应用。

它不仅支持标准的 Markdown 富文本编辑与渲染，还深度集成了 OpenAI (LLM) 能力。

用户可以在本地通过 Room 数据库 高效管理海量笔记，并利用 AI 自动生成内容摘要与智能标签。项目旨在探索端侧 AI 与传统工具类应用的结合，同时在工程上实现了模块化设计与极致的性能优化。

核心功能点：

☆ 核心笔记功能：

支持笔记的新增、编辑、删除及批量管理。

支持多标签，支持标签编辑、删除等基本能力

支持 Markdown 富文本渲染（加粗、代码块高亮、表格、列表等）。

支持“编辑”与“预览”模式的一键切换。

☆ AI 智能辅助：

支持 AI 摘要生成：一键调用大模型总结长笔记核心内容。

☆ 数据管理：

本地持久化：基于 SQLite 数据库存储。

全文本搜索：支持对标题、正文、标签的毫秒级检索。

☆ 交互与体验：

采用 Material Design 设计语言，使用圆角、描边与大色块区分功能区，界面简洁直观。
支持深色/浅色模式适配。

3. 技术栈

分类（Category）	核心技术（Technology）	应用场景与说明（Description）
基础架构	MVVM	采用 Model-View-ViewModel 模式，实现 UI 与业务逻辑解耦，提升代码可维护性
开发语言	Java+Kotlin	全项目采用 Java+Kotlin 编写，深度利用扩展函数、Lambda 表达式等现代化特性。
UI 构建	Android Native (XML)	基于 Material Design 规范，使用 ConstraintLayout 构建自适应布局。
异步并发	Coroutines & Flow	使用协程处理数据库读写与网络请求，利用 Flow 实现响应式数据流
本地存储	Jetpack Room	封装 SQLite 数据库，实现毫秒级本地全文本搜索
网络通信	OkHttp 4	负责与 AI 模型 API 进行 HTTP 通信，配置拦截器处理鉴权与日志。
富文本引擎	Markwon	高性能 Markdown 解析库，定制 SpanFactory 实现代码高亮与特殊样式渲染
依赖注入	Manual DI / AppContainer	采用手动依赖注入模式管理 Repository 与 Database 单例，确保资源统一管理

4. 作业完成情况

阶段一	基础功能 (CRUD, Room 数据库, 标签管理)
阶段二	进阶交互 (Markdown 渲染, 预览模式, Git 协作)
阶段三	AI 集成 (对接 API, 生成摘要)
阶段四	性能优化 (启动优化, 内存检测)

二、系统架构设计

本项目采用 Google 推荐的 MVVM (Model-View-ViewModel) 架构模式进行开发，遵循“单一数据源 (SSOT)”和“关注点分离”的设计原则。系统整体分为三层：

- 1. 表现层 (UI Layer)：负责界面展示与用户交互。

2. 领域层/状态层 (ViewModel Layer): 负责持有 UI 状态并处理业务逻辑。
3. 数据层 (Data Layer): 负责数据的获取、存储与分发。

2.1 表现层 (UI Layer)

核心职责：只负责“怎么画界面”和“捕获用户点击”，绝对不写复杂的逻辑判断。技术实现：

- 使用 XML Layouts 构建界面，结合 ConstraintLayout 实现复杂布局的自适应。
- 使用 Activity 作为 UI 容器（如 MainActivity, NoteDetailActivity）。

◦ 使用 RecyclerView 配合自定义的 MarkdownContentAdapter 实现高性能的笔记列表与预览渲染。

关键亮点：

- 双模式渲染：在 NoteDetailActivity 中实现了“编辑模式”（EditText）与“预览模式”（Markwon 渲染）的无缝切换。
- 状态观察：UI 层不主动去“拉”数据，而是通过观察 ViewModel 中的 StateFlow 或 LiveData 被动响应界面更新。

2.2 状态层 (ViewModel Layer)

核心职责：它是 UI 的“大脑”。它从数据层拿数据，处理好后交给 UI 层。

技术实现：

◦ 继承 androidx.lifecycle.ViewModel，确保数据在屏幕旋转等配置变更时不会丢失。

◦ NoteListViewModel：管理笔记列表数据，处理搜索关键词过滤逻辑。

◦ NoteDetailViewModel：管理单篇笔记的加载、保存、删除以及 AI 摘要生成的异步任务状态。

并发处理：

◦ 利用 Kotlin Coroutines (viewModelScope) 启动协程，确保耗时操作（如数据库读写、AI 网络请求）不阻塞主线程。

2.3 数据层 (Data Layer)

核心职责：App 的“仓库管理员”。ViewModel 想要数据，找它就对了，

ViewModel 不需要知道数据是来自手机还是网络。

Repository 模式：

- 创建 NoteRepository 类，作为数据的唯一访问入口。

数据源 A：本地数据库 (Local Data Source)： ◦ 基于 Jetpack Room (SQLite)

实现。◦ 定义 NoteDao 接口，提供 insert, delete, update, query 等 CRUD 操作。

- 使用 Flow<List<Note>> 实现响应式数据流，当数据库发生变化时，自动通知上层更新。

数据源 B：网络服务 (Remote Data Source)：

- 基于 OkHttp 4 封装 AIService。

- 负责与 OpenAI 接口进行通信，处理 API Key 鉴权、JSON 解析与错误重试。

核心技术说明：

模块/功能	选型方案	选型理由
架构模式	MVVM	相比 MVC/MVP，MVVM 能更好地解耦 UI 与逻辑，且配合 Jetpack 组件库（ViewModel/LiveData）开发效率最高。
异步编程	Kotlin Coroutines	相比 RxJava 或 AsyncTask，协程代码更简洁（用同步的方式写异步代码），且轻量级，无回调地狱。
数据库	Jetpack Room	Google 官方封装的 ORM 库，提供了编译期 SQL 检查，且完美支持 Kotlin Flow，比原生 SQLite 易用且安全。
网络库	OkHttp	行业标准的 HTTP 客户端，支持拦截器（Interceptors）处理公共 Header（如 Authorization），扩展性强。
Markdown 渲染	Markdown	Android 平台上性能最好、扩展性最强的 Markdown 库，支持自定义 SpanFactory（实现了你的加粗下划线需求）。

目录架构设计



三、数据库详细设计

3.1 数据存储模块

1. 本系统主要包含两个核心实体：

Note (笔记): 存储笔记的核心内容、AI 摘要及元数据。

Tag (标签): 存储标签名称与颜色信息

2. 笔记表

字段名 (Field)	类型 (Type)	约束 (Constraints)	说明 (Description)
id	Long	Primary Key, AutoGenerate	笔记唯一标识符

title	String	Not Null	笔记标题
content	String	Not Null	笔记正文
createAt	Long	Not Null	创建时间戳
updateAt	Long	Not Null	最后修改时间戳

代码实现

```
//这个是名为notes的table
package com.example.noteai.data
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "notes")
data class Note(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val title: String,
    val content: String,
    val createdAt: Long = System.currentTimeMillis(),
    val updatedAt: Long = System.currentTimeMillis()
)
```

3. 数据访问对象的设计

接口名: noteDao

核心方法定义:

响应式查询: 返回 Flow<List<NoteWeightTags>>, 当数据库数据变化时, UI 会自动更新, 无需手动刷新。

搜索: 利用 SQL LIKE 语句实现对标题和内容的模糊搜索。

去重: 利用 IGNORE 避免 tag 重复

挂起函数: 写入操作 (增删改) 使用 upsert, 强制在协程中执行, 防止阻塞主线程。

代码实现:

```

@Dao
interface NoteDao {
    //这个transaction是为了让我们在查询的时候note和tags一起查就避免了数据不一致这种情况
    //用这个flow主要是为了自动listen db的变化，一有这个数据的更新我们的UI需要自动地刷新
    @Transaction
    @Query("SELECT * FROM notes ORDER BY updatedAt DESC")
    fun getAllNotesWithTags(): Flow<List<NoteWithTags>>

    @Transaction
    @Query(
        "SELECT * FROM notes WHERE title LIKE '%' || :query || '%' OR content LIKE '%' || :query"
    )
    fun searchNotesWithTags(query: String): Flow<List<NoteWithTags>>

    //用java要重新开线程吧，就用suspend协程了一下
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertNote(note: Note): Long

    @Update
    suspend fun updateNote(note: Note)

    @Delete
    suspend fun deleteNote(note: Note)

    //用ignore来避免tag的名字重复
    @Insert(onConflict = OnConflictStrategy.IGNORE)
    suspend fun insertTags(tags: List<Tag>): List<Long>

    @Query("SELECT * FROM tags WHERE name IN (:names)")
    suspend fun getTagsByNames(names: List<String>): List<Tag>

    //用ignore来避免tag的名字重复
    @Insert(onConflict = OnConflictStrategy.IGNORE)
    suspend fun insertTags(tags: List<Tag>): List<Long>

    @Query("SELECT * FROM tags WHERE name IN (:names)")
    suspend fun getTagsByNames(names: List<String>): List<Tag>

    //获取所有标签，按名字排序
    @Query("SELECT * FROM tags ORDER BY name ASC")
    fun getAllTags(): Flow<List<Tag>>

    //删除标签
    @Delete
    suspend fun deleteTag(tag: Tag)

    @Transaction
    @Query("SELECT * FROM notes WHERE id = :noteId LIMIT 1")
    suspend fun getNoteWithTags(noteId: Long): NoteWithTags?

    //嗯这个其实可选吧，写这个是为了保证note id和tag id每对的唯一性，重复插入同样一对noteid和tagid可以新
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCross(cross: List<NoteTagCross>)

    @Query("DELETE FROM note_tag_cross WHERE noteId = :noteId")
    suspend fun deleteCrossForNote(noteId: Long)

    //我写upsert可能有点奇怪，其实意思是update+insert。如果note的id是0说明是新建，否则意思是更新已经存在
    //处理了note和tag的关系就啥时候update/啥时候insert
    @Transaction
    suspend fun upsertNoteWithTags(note: Note, tags: List<Tag>): Long {
        val persistedNoteId = if (note.id == 0L) {
            insertNote(note)
        } else {
            updateNote(note)
            note.id
        }
    }

```

```

//如果有update的话，先删除以前的notetag关系，然后重新建立新的关系
deleteCrossForNote(persistedNoteId)
if (tags.isEmpty()) return persistedNoteId
insertTags(tags)
//根据tag的名字查出tag所对应的id，然后创建cross table的记录
val savedTags = getTagsByNames(tags.map { it.name })
val cross = savedTags.map { tag -> NoteTagCross(noteId = persistedNoteId, tagId = tag.id)
insertCross(cross)
return persistedNoteId
}

//删除笔记的时候也要把相关的tag关系删掉
@Transaction
suspend fun deleteNoteWithTags(note: Note) {
    deleteCrossForNote(note.id)
    deleteNote(note)
}

```

4. 数据库配置

定义 Room 数据库的入口点，采用单例模式防止多实例导致的数据竞争问题。

```

//这个就是database
package com.example.noteai.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(
    entities = [Note::class, Tag::class, NoteTagCross::class],
    version = 2, //添加了tag颜色字段
    exportSchema = false
)

abstract class NoteDatabase : RoomDatabase() {

    abstract fun noteDao(): NoteDao

    //这个是为了保证整个app只有一个数据库实例
    companion object {
        //这个volatile则是为了保证多线程环境下instance的可见性
        @Volatile
        private var instance: NoteDatabase? = null

        //double check一下避免重复创建数据库
        fun getInstance(context: Context): NoteDatabase {
            return instance ?: synchronized(this) {
                instance ?: Room.databaseBuilder(
                    context.applicationContext,
                    NoteDatabase::class.java,
                    "note_ai.db"
                ).fallbackToDestructiveMigration() //失败就直接删除重建好了
                .build()
                .also { instance = it }
            }
        }
    }
}

```

3.2 UI 与 Markdown 渲染模块

1. 双模式交互:

编辑模式: 原生 **EditText**，专注于文本输入，支持快捷插入符号。

预览模式: 使用 **RecyclerView + MarkdownContentAdapter**

2. 自定义渲染 (MarkdownHelper):

利用 **Markwon** 插件系统。

样式增强: 自定义 **Spans** 实现加粗字体变黑+下划线（已实现）。

代码块: 定制背景色和字体。

性能优化: 采用 **RecyclerView** 对长文本进行“分块渲染”或复用，避免一次性渲染超长 **Markdown** 导致的 **OOM** (内存溢出) 或掉帧。

3.3 AI 服务集成设计

本项目集成了大语言模型 (**LLM**) 能力，旨在通过 **AI** 辅助用户更高效地管理笔记。考虑到移动端性能与网络环境，采用 **Retrofit/OkHttp** 直连 **OpenAI** 接口 的方案，利用 **Kotlin** 协程实现非阻塞式的流式交互。

1. AI 模块架构设计

UI 层 (NoteDetailActivity): 仅负责触发请求（如点击“生成摘要”按钮）和展示结果。

ViewModel 层 (NotesViewModel): 管理协程作用域 (**viewModelScope**)，处理加载状态 (**Loading/Success/Error**)。

Service 层 (OpenAiClient): 封装 **HTTP** 请求细节，处理鉴权、**JSON** 解析与超时重试。

2. 客户端

对外暴露的 **AI** 能力接口，不依赖 **Android UI / Room**，只用普通字符串，这样就和其他功能解耦

代码实现

```
interface AiClient {

    //用 OpenAI 给笔记生成 1-2 句摘要，保持原文语言
    suspend fun summarizeNote(
        title: String,
        content: String
    ): String

    //根据笔记内容生成主题标签（1-3 个词），返回的是纯文本标签列表。
    suspend fun suggestTopics(
        title: String,
        content: String
    ): List<String>
}
```

四、性能优化与工程质量

4.1 数据库优化

基于 **NoteDao** 的代码实现，我们采用了以下策略来保证数据读写的高效性：

1. 事务管理 (Transactions):

策略: 在涉及多表操作（如同时插入笔记和标签、更新关联表）时，使用

@Transaction 注解。

代码体现: 在 **upsertNoteWithTags** 方法中，我们将插入笔记、删除旧关

联、插入新标签、建立新关联这 4 步操作封装在一个事务中。

收益: 保证了数据的原子性（要么全成，要么全败），同时相比多次开启数据库连接，写入性能提升。

2. 响应式数据流 (Reactive Streams):

策略: 查询操作（如 `getAllNotesWithTags`）直接返

Flow<List<NoteWithTags>>

收益: 利用 **Room** 的原生 **Flow** 支持，当底层数据发生变化时，**UI** 层会自动收到通知并刷新，避免了手动轮询数据库造成的资源浪费。

3. 异步非阻塞 I/O:

策略: 所有写入 (**Insert**, **Update**, **Delete**) 操作均定义为 **suspend** 函数。

收益: 强制在 **Kotlin** 协程的 **IO** 调度器中执行，彻底杜绝了主线程操作数据库导致的 **ANR (Application Not Responding)** 风险。

4.2 UI 渲染

1. 列表渲染优化 (RecyclerView):

DiffUtil: 在 **NoteAdapter** 中使用 **DiffUtil.ItemCallback** 替代传统的 **notifyDataSetChanged()**。

原理: 仅计算数据变化的部分（如只更新了某一行），触发局部刷新，大幅减少布局重绘开销。

3. Markdown 渲染优化:

策略: **Markwon** 解析是耗时操作。在长文本场景下，通过在后台线程预解析 **Markdown** 语法树，再回到主线程渲染 **Spans**，防止界面卡顿。

4.3 工程协作规范

本项目严格遵循 **Git Flow** 分支管理规范，确保多人协作时的代码稳定性。

1. 分支策略:

main: 主分支，仅允许合并经过测试的稳定代码。

feature/xxx: 功能分支（如 **feature/markdown-preview**），开发完成后通过 **PR** 合并。

2. 代码审查 (Code Review):

所有代码合并前必须发起 **Pull Request (PR)**。

团队成员针对代码逻辑、命名规范及潜在 **Bug** 进行评论与修正（如解决 **build.gradle.kts** 冲突）。

3. 版本控制:

提交信息 (Commit Message) 遵循 **Conventional Commits** 规范（如 **Feat: Add AI summary, Fix: Database transaction error**），生成清晰的变更日志。